Documento explicativo - Proyecto Intermedio

Adrián Isasi Martínez

> 🖊 **El documento está escrito en *markdown* (.md), es un formato légible sin previsualizador, pero hay aplicaciones que muestran el texto más bonito.**
>
> Mi intención es subir un .pdf a la entrega, pero si no, esto se puede leer desde GitHub fácilmente (https://github.com/crookie-school/SimulacionFisicaVideojuegos/blob/main/document_explicativo.md).

Instrucciones para ejecutar

No hay un juego todavía, mi juego usará principalmente rigidbodies, el proyecto solo contiene el código de las prácticas. Para ejecutarlo, se debe abrir el proyecto con VS2022 (puede que funcione con otras versiones de VS, pero no lo he probado), y presionar la flechita verde para ejecutar la aplicación. Se puede seleccionar otro perfil de compilación.

> ⚠ **Solo se ejecuta en DEBUG, intenté averiguar porqué no compila en RELEASE, pero no lo conseguí.**

Cada escena (asignada a 1, 2, 3 o 4) representa una "demo" de las prácticas correspondientes. La primera escena es la vacía que se pone por defecto. Para cambiar de escena presiona el número de la escena a cambiar en el teclado. Para mover la cámara usa [W] [A] [S] [D] y el ratón.
Para ver lo que se puede hacer en cada escena mira la siguiente sección que detalla todas las teclas disponibles.

Interfaz gráfica

He implementado una una interfaz gráfica para que se pueda interactuar con las prácticas de la forma más intuitiva posible.
Además esto me permitía añadir muchísimos más ajustes de forma cómoda.
Para ello he utilizado la librería ImGui (https://github.com/ocornut/imgui/), que es una de las más famosas. Simplemente he copiado el código necesario en una carpeta y lo he incluido en la compilación.
Todo el código necesario para dibujar la UI está dentro de funciones con nombres como

'drawGui' o 'showUI'. He intentado comentar un poco lo que hace cada cosa, pero como tampoco tiene mucho que ver con la asignatura tampoco es super explicativo.
Este código se puede ignorar, no afecta en nada a la simulación, es solo para permitir modificar los elementos de la simulación en tiempo real de forma cómoda.

El uso debería ser intuitivo, hay un panel principal arriba que incluye un desplegable para cambiar de escenas, la escena actual y el framerate.
En cada escena saldrá un panel pertinente en el que se pueden modificar los atributos de la escena.

Escena 1: La interfaz permite seleccionar un proyectil y configurarlo. Además se pueden crear nuevos proyectles o borrar los proyectiles que no queramos.

Escena 2: La interfaz permite generar un número de partículas, activar la gravedad, configurar el generador y las propiedades del generador.
La escena 2 tiene muchas opciones bajo los desplegables 'Main generation settings' y 'Children generation settings'.
'Main generatino settings' contiene la configuración de generación de las partículas padre. He puesto que se puedan crear nuevas configuraciones de particulas.
'Children generation settings' contiene la configuración de generación del las partículas hija, pero solo lo muestra si está activada la generación de hijas al morir ('Spawn children').

Escena 3: La interfaz permite modificar el generador de partículas (igual que en la escena 2), activar/desactivar y configurar cada generador de fuerzas. Las opciones de cada cosa están bajo los sus desplegables.

Escena 4: Esta escena no entra en el proyecto intermedio, pero como la tenía hecha me he visto obligado a ponerle también una interfaz.

Teclas disponibles

Cada escena es una demo de dicha práctica. A continuación se describen las teclas disponibles en cada escena.

> **'[]' significa que hay que presionar la tecla con el carácter .**

Cámara:

- **Mover**: [W] [A] [S] [D]
- **Girar**: [Ratón]

Cambiar de escena:

- **Escena 1** (demo de práctica 1): [1]

- **Escena 2** (demo de práctica 2): [2]
- **Escena 3** (demo de práctica 3): [3]
- **Escena 4** (demo de práctica 4): [4]

Escena 1:

- **Disparar**: [Espacio]

Requisitos de programación

Requisito A.1) -> "Particle.cpp" 49 - 94

Requisito A.2) -> "Particle.h" 52 atributo de la inversa de la masa, "P1_Scene.h" Las distintas balas tienen masas distintas. Además se puede cambiar la masa para ver los cambios en la simulación de proyectiles.

Requisito A.3) -> "Particle.h" 48 - 49 métodos para aplicar o resetear fuerzas. "Particle.cpp" 49 - 94 aplicación de las fuerzas en los distintos integradores

Requisito B) -> "Projectile.h" y "Projectile.cpp", en la escena 1 ("P1_Scene.h") se puede ver que hay 3 tipos de proyectiles y la posibilidad de modificar sus propiedades con 'offsets'.

Requisito C) -> "ParticleSystem.h" y "ParticleSystem.cpp", básicamente es un contenedor de partículas ("Particle.h"). Para mantener una abstracción limpia, NO es un contenedor de fuerzas, y NO gestiona las fuerzas, eso se hace por separado para mantener la modularidad del código. Contiene y mantiene un conjunto de partículas, y expone una interfaz segura al mismo. Los generadores de fuerzas usan esta interfaz para aplicar la fuerza a las partículas. En ParticleSystem se usa una cola de espera para añadir partículas. Esto se debe a la implementación del opcional, que permite que se creen partículas cuando otras mueren. Mantiene la integridad del vector de partículas aunque se añadan partículas mientras se itera por él.

Requisito D.1) -> "ParticleGenerator.h" 7 - 24, La configuración de generación del generador está contenida en el struct *ParticleGenerationConfig* que define los atributos usados para generar partículas por el generador. Previamente usaba una partícula como modelo, pero lo cambie a campos libres porque la partícula no tenía todos los atributos necesarios y era mucho menos legible qué era configurable y qué no.

Requisito D.2) -> "ParticleGenerator.h" 12 - 13, permite configurar parámetros de generación (como la posición en este caso), y "ParticleGenerator.cpp" 13 - 37 función de ayuda que se usa para conseguir un punto aleatorio según la configuración del usuario. Se puede escoger qué tipo de función usar al crear el objeto generador, pasando como parámetro de construcción la función de números aleatorios ("ParticleGenerator.h" 29)

Requisito D.3) -> Yo he hecho una implementación distinta (en mi opinión es más claro y desacoplado). En vez de que un sistema de partículas contenga un generador, uno puede indicarle a un generador cualquiera en qué sistema de partículas debe generar partículas. Esto hace que la dependencia entre ambos sea unidireccional y no bidireccional. Además permite que un generador cambie el sistema en el que genera partículas, o la forma en las que las genera sin problemas de referencias. Un ejemplo de implementación es "P2_Scene.h", donde se puede configurar el generador en tiempo real mientras genera partículas para ver el resultado de los cambios.

Requisito E.1) -> "ForceGenerator.h" 22 - 34 (el generador de gravedad). La única función del generador de fuerzas es, dada una párticula y una configuración, aplicar la fuerza correcta a dicha partícula. La fuerza aplicada a cada partícula es calculada por separado, pues consulta las propiedades de cada partícula. Me parece muy importante aclarar que la partícula NO sabe nada de los generadores de fuerzas, esto es importante que sea así porque queremos desacoplar la lógica de las partículas de los generadores de fuerzas.

Requisito E.2) -> "P3_Scene.h" 60 - 78 Una partícula puede verse afectada por más de un generador de fuerzas si se ordena que distintos generadores de fuerzas afecten al mismo sistema de partículas (o la misma partícula).

Requisito E.3) -> "ForceGenerator.h" o "P3_Scene.h" Los generadores de fuerzas no guardan ninguna referencia a ningún sistema. Solo saben aplicar una fuerza a una partícula dada (o sistema, usando los iteradores). Esto permite que cada sistema reciba fuerzas totalmente distintas.

Requisito E.4) -> "P3_Scene.h" 60 - 78 Se puede observar que se han implementado las fuerzas del viento, tornados, gravedad, (para flotación mejor ver P4_Scene aunque no sea de esta entrega), y explosiones.

Requisitos mínimos de ejecución

1. "P1_Scene.h", la escena 1 permite disparar distintos proyectiles con distintas propiedades y probar con distintas configuraciones. Todos los disparos salen desde la cámara.
2. "P3_Scene.h", Se puede activar cada generador con la UI, pudiendo tener activos todos, algunos o ningún generador.